Политыко Илья Андреевич 321701

Лабораторная работа 6

Численное решение задачи Коши для ОДУ первого порядка и их систем

```
Clear[x, y]
очистить
f[x_, y_] = Cos[x + y] - x - y;
            косинус
```

```
In[•]:= a = 0; b = 1; h1 = 0.1; h2 = 0.05; n1 = Floor[(b - a)/h1]; n2 = Floor[(b - a)/h2]; x0 = 0; y0 = 0;
                                                  округление вниз        округление вниз
```

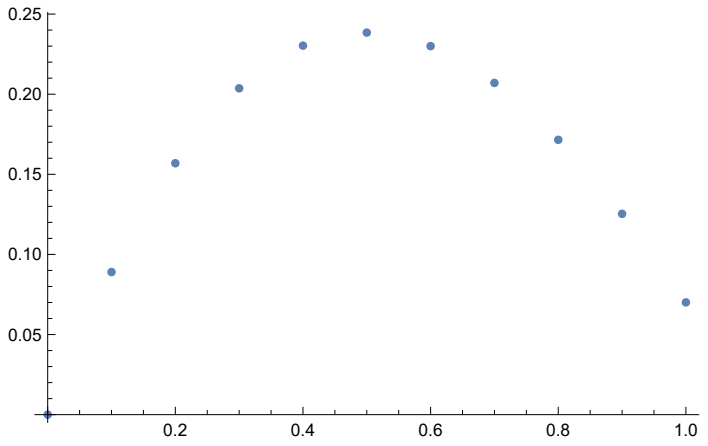
```
In[•]:= x0 = 0; y0 = 0;
        eulk1 = Table[{x0, y0} =
                таблица значений
           {x0 + h1, y0 + h1 / 2 * (f[x0, y0] + f[x0 + h1, y0 + h1 * f[x0, y0]])}, {i, 0, n1 - 1}]
Out[•]=
        {{0.1, 0.0890033}, {0.2, 0.156893}, {0.3, 0.20367}, {0.4, 0.230311}, {0.5, 0.238435},
         {0.6, 0.22999}, {0.7, 0.207021}, {0.8, 0.171515}, {0.9, 0.125311}, {1., 0.0700556}}
```

```
In[•]:= eulk1 // Column
                 колонка
Out[•]=
        {0.1, 0.0890033}
        {0.2, 0.156893}
        {0.3, 0.20367}
        {0.4, 0.230311}
        {0.5, 0.238435}
        {0.6, 0.22999}
        {0.7, 0.207021}
        {0.8, 0.171515}
        {0.9, 0.125311}
        {1., 0.0700556}
```

```
In[•]:= eulk1 = Prepend[eulk1, {0, 0}]
                 добавить в начало
Out[•]=
        {{0, 0}, {0.1, 0.0890033}, {0.2, 0.156893},
         {0.3, 0.20367}, {0.4, 0.230311}, {0.5, 0.238435}, {0.6, 0.22999},
         {0.7, 0.207021}, {0.8, 0.171515}, {0.9, 0.125311}, {1., 0.0700556}}
```

In[∘]:=
```
gr1 = ListPlot[eulk1]
```
диаграмма разброса

Out[∘]=



In[∘]:=
```
x0 = 0; y0 = 0;
eulk2 = Table[{x0, y0} =
    {x0 + h2, y0 + h2 / 2 * (f[x0, y0] + f[x0 + h2, y0 + h2 * f[x0, y0]])}, {i, 0, n2 - 1}]
```
таблица значений

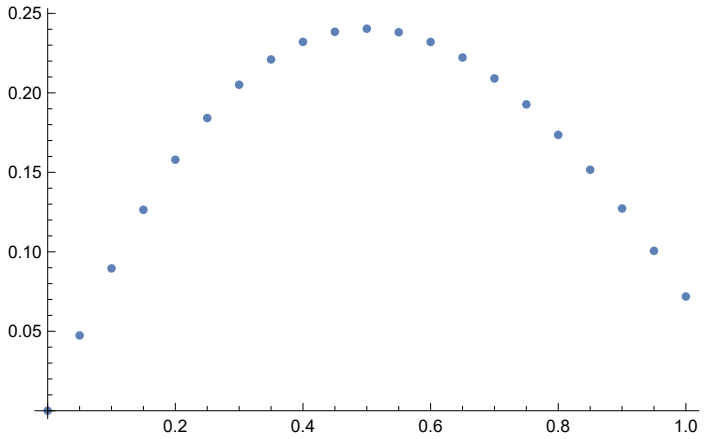
Out[∘]=
```
{{0.05, 0.0473751}, {0.1, 0.0895549}, {0.15, 0.126421}, {0.2, 0.157935},
 {0.25, 0.184132}, {0.3, 0.205111}, {0.35, 0.221019}, {0.4, 0.232048},
 {0.45, 0.238416}, {0.5, 0.240365}, {0.55, 0.238146}, {0.6, 0.23202},
 {0.65, 0.222244}, {0.7, 0.209074}, {0.75, 0.192757}, {0.8, 0.173532},
 {0.85, 0.151624}, {0.9, 0.127247}, {0.95, 0.100603}, {1., 0.07188}}
```

In[∘]:=
```
eulk2 = Prepend[eulk2, {0, 0}]
```
добавить в начало

Out[∘]=
```
{{0, 0}, {0.05, 0.0473751}, {0.1, 0.0895549}, {0.15, 0.126421}, {0.2, 0.157935},
 {0.25, 0.184132}, {0.3, 0.205111}, {0.35, 0.221019}, {0.4, 0.232048},
 {0.45, 0.238416}, {0.5, 0.240365}, {0.55, 0.238146}, {0.6, 0.23202},
 {0.65, 0.222244}, {0.7, 0.209074}, {0.75, 0.192757}, {0.8, 0.173532},
 {0.85, 0.151624}, {0.9, 0.127247}, {0.95, 0.100603}, {1., 0.07188}}
```

In[∘]:=
```
gr2 = ListPlot[eulk2]
```
диаграмма разброса

Out[∘]=



```
(*б)Метод Рунге-Кутта*)
```

In[•]:=
```
Clear[x, y]
очистить
f[x_, y_] = Cos[x + y] - x - y;
            косинус
a = 0;
b = 1;
x0 = 0;
y0 = 0;
h1 = 0.1;
h2 = 0.05;
n1 = Floor[(b - a)/h1];
     округление вниз
n2 = Floor[(b - a)/h2];
     округление вниз
```

In[•]:=
```
sol1 = List[{x0, y0}];
       список
```

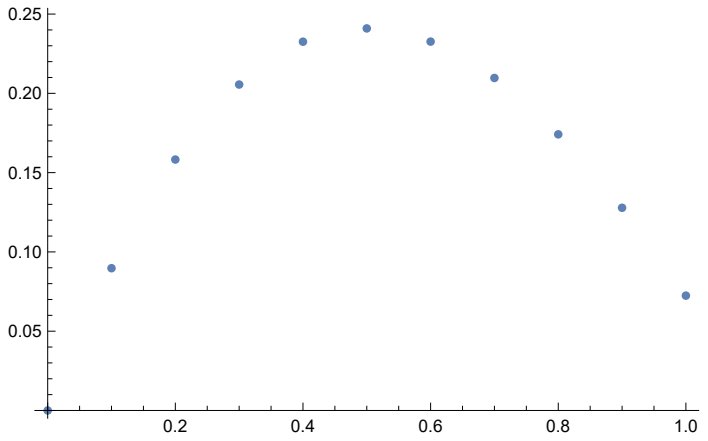
In[•]:=
```
x = x0; y = y0;
For[k = 1, k < n1 + 1, k++,
цикл ДЛЯ
 k1[x_, y_] = h1 * f[x, y];
 k2[x_, y_] = h1 * f[x + h1 / 2, y + k1[x, y] / 2];
 k3[x_, y_] = h1 * f[x + h1 / 2, y + k2[x, y] / 2];
 k4[x_, y_] = h1 * f[x + h1, y + k3[x, y]];
 x = x + h1;
 y = y + (k1[x, y] + 2 * k2[x, y] + 2 * k3[x, y] + k4[x, y]) / 6;
 sol1 = Append[sol1, {x, y}]]
        добавить в конец
```

In[•]:=
```
sol1
```

Out[•]=
```
{{0, 0}, {0.1, 0.0897242}, {0.2, 0.158253},
 {0.3, 0.20555}, {0.4, 0.232576}, {0.5, 0.240949}, {0.6, 0.232634},
 {0.7, 0.209694}, {0.8, 0.17414}, {0.9, 0.12783}, {1., 0.0724284}}
```

In[•]:=
```
gr3 = ListPlot[sol1]
```
диаграмма разброса

Out[•]=




```
(*с помощью функций DSolve и NDSolve,построить графики.*)
```
решить ··· численно решить ДУ

In[•]:=
```
Clear[sol2]
```
очистить
```
sol2 = List[{x0, y0}];
```
список

In[•]:=
```
x = x0; y = y0;
For[k = 1, k < n2 + 1, k++,
 k1[x_, y_] = h2 * f[x, y];
 k2[x_, y_] = h2 * f[x + h2 / 2, y + k1[x, y] / 2];
 k3[x_, y_] = h2 * f[x + h2 / 2, y + k2[x, y] / 2];
 k4[x_, y_] = h2 * f[x + h2, y + k3[x, y]];
 x = x + h2;
 y = y + (k1[x, y] + 2 * k2[x, y] + 2 * k3[x, y] + k4[x, y]) / 6;
 sol2 = Append[sol2, {x, y}]]
```
цикл ДЛЯ

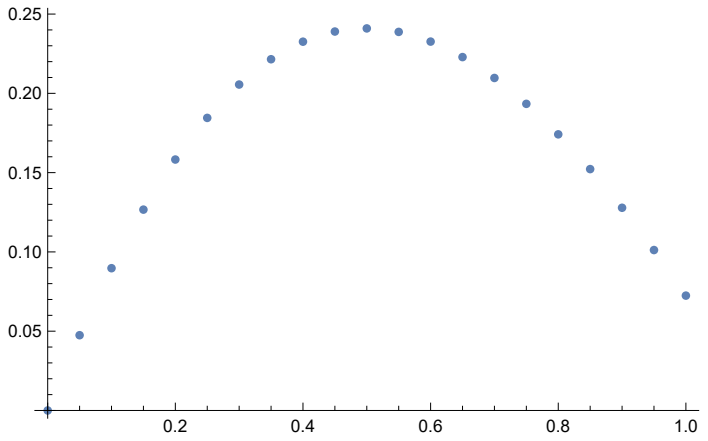
добавить в конец

In[•]:=
```
sol2
```

Out[•]=
```
{{0, 0}, {0.05, 0.047462}, {0.1, 0.089725}, {0.15, 0.126669}, {0.2, 0.158255},
 {0.25, 0.184517}, {0.3, 0.205552}, {0.35, 0.22151}, {0.4, 0.232579},
 {0.45, 0.23898}, {0.5, 0.240953}, {0.55, 0.238753}, {0.6, 0.232638},
 {0.65, 0.222868}, {0.7, 0.209698}, {0.75, 0.193377}, {0.8, 0.174144},
 {0.85, 0.152224}, {0.9, 0.127834}, {0.95, 0.101173}, {1., 0.0724322}}
```

In[∘]:=
```
gr4 = ListPlot[sol2]
```
диаграмма разброс

Out[∘]=



In[∘]:=
```
Clear[y, x] (*Очистка предыдущих определений*)
```
очистить
```
eqn = y'[x] == -x + Cos[x + y[x]] - y[x];
```
косинус
```
initialCondition = y[0] == 0;
solutionDSolve = DSolve[{eqn, initialCondition}, y, x];
```
решить дифференциальные уравнения
```
y1[x_] = y[x] /. Flatten[solutionDSolve]
```
уплостить
```
Print["Аналитическое решение: ", solutionDSolve];
```
печатать

ReplaceAll: $\left\{\text{Solve}\left[\int_0^x (-1 - \text{Power}[\ll 2 \gg]) \, dK[1] + \int_0^{y[x]} (-\text{Power}[\ll 2 \gg] - \ll 1 \gg[\ll 2 \gg]) \, dK[2] == 0, y[x]\right]\right\}$ is neither a list of replacement rules nor a valid dispatch table, and so cannot be used for replacing.

Out[∘]=

$$y[x] \;/.\; \text{Solve}\left[\int_0^x \left(-1 - \frac{1}{-1 - \text{Cos}[K[1] + y[x]] + K[1] + y[x]}\right) dK[1] + \int_0^{y[x]} \left(-\frac{1}{-1 + x - \text{Cos}[x + K[2]] + K[2]} - \int_0^x \frac{1 + \text{Sin}[K[1] + K[2]]}{(-1 - \text{Cos}[K[1] + K[2]] + K[1] + K[2])^2} dK[1]\right) dK[2] == 0, y[x]\right]$$

Аналитическое решение:

$$\text{Solve}\left[\int_0^x \left(-1 - \frac{1}{-1 - \text{Cos}[K[1] + y[x]] + K[1] + y[x]}\right) dK[1] + \int_0^{y[x]} \left(-\frac{1}{-1 + x - \text{Cos}[x + K[2]] + K[2]} - \int_0^x \frac{1 + \text{Sin}[K[1] + K[2]]}{(-1 - \text{Cos}[K[1] + K[2]] + K[1] + K[2])^2} dK[1]\right) dK[2] == 0, y[x]\right]$$

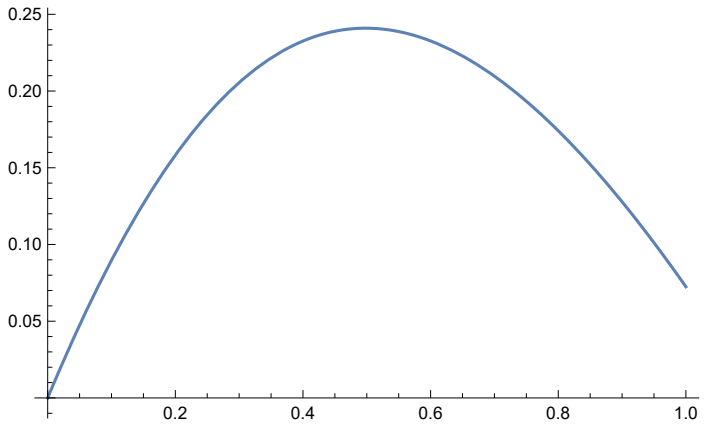
In[∘]:=
```
Clear[y, x] (*Очистка предыдущих определений*)
очистить
eqn = y'[x] == -x + Cos[x + y[x]] - y[x];
                    косинус
initialCondition = y[0] == 0;
solutionNDSolve = NDSolve[{eqn, initialCondition}, y, {x, 0, 1}]
                  численно решить ДУ
```

Out[∘]=

{{y → InterpolatingFunction[Domain: {{0., 1.}} Output: scalar]}}

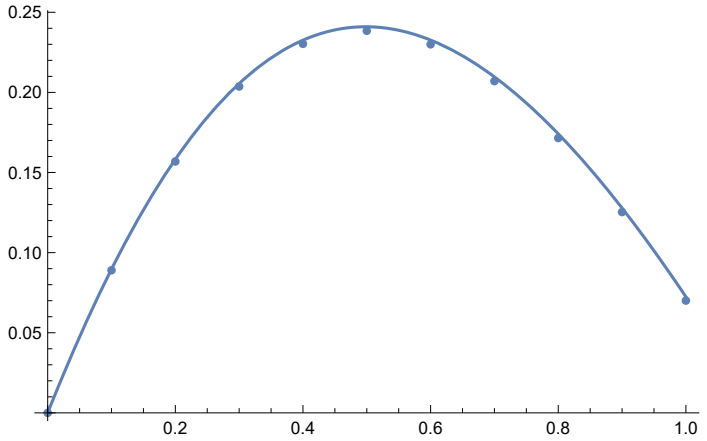
In[∘]:=
```
gr5 = Plot[Evaluate[y[x] /. solutionNDSolve], {x, 0, 1}]
      гр···  вычислить
```

Out[∘]=



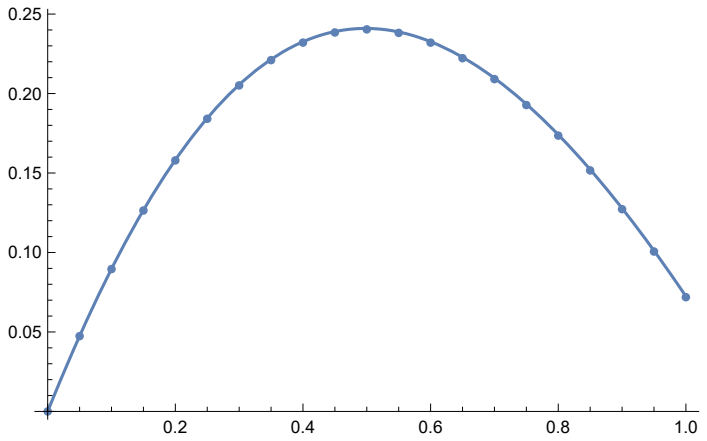
In[∘]:=
```
Show[gr1, gr5, ImageSize → Medium]
показать      размер изоб··· средний
```

Out[∘]=



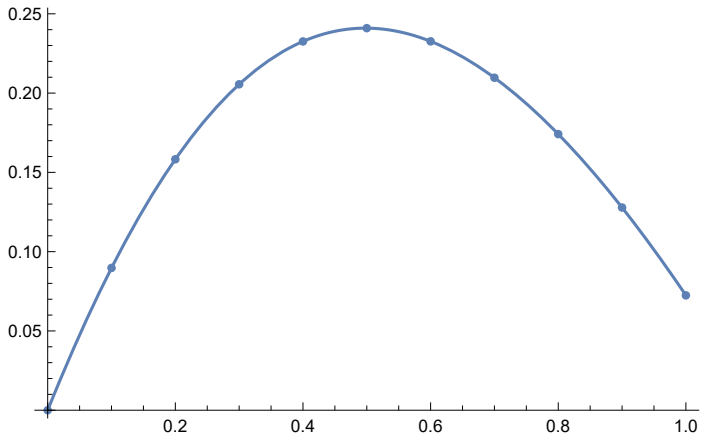
In[∘]:=
```
Show[gr2, gr5, ImageSize → Medium]
```
показать размер изоб··· средний

Out[∘]=



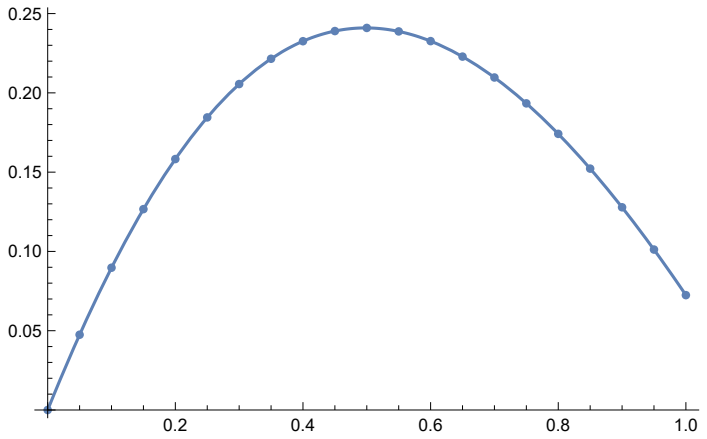
In[∘]:=
```
Show[gr3, gr5, ImageSize → Medium]
```
показать размер изоб··· средний

Out[∘]=



In[∘]:=
```
Show[gr4, gr5, ImageSize → Medium]
```
показать размер изоб··· средний

Out[∘]=

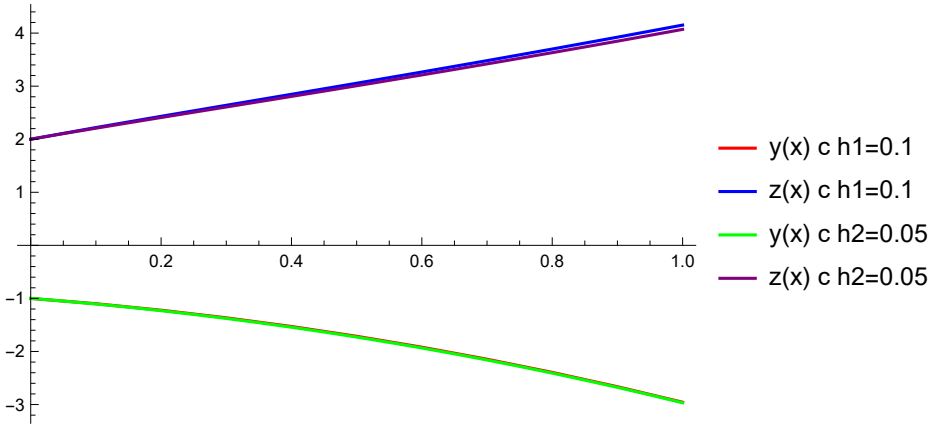


2. Решить задачу Коши для системы двух дифференциальных уравнений на отрезке [0, 1]

а) Метод Эйлера

```
Clear[y, z]
очистить
a = 0;  b = 1;     y0 = -1;    z0 = 2;    h1 = 0.1;    h2 = 0.05;
f[y_, z_] := 1 - z;
g[y_, z_, x_] := Log[x + 1] - (2 * y) / ((x + 1)^2);
                 натуральный логарифм
eulerMethod[{y0_, z0_}, h_, n_] :=
 Module[{x = a, y = y0, z = z0, result = {{x, y, z}}},
 программный модуль
  AppendTo[result, {x, y, z}];
  добавить в конец к
  For[i = 1, i ≤ n, i++,
  цикл ДЛЯ
   y = y + h * f[y, z];
   z = z + h * g[y, z, x];
   x = x + h;
   AppendTo[result, {x, y, z}];
   добавить в конец к
  ];
  result
 ];

n1 = Floor[(b - a) / h1];
     округление вниз
n2 = Floor[(b - a) / h2];
     округление вниз
euler1 = eulerMethod[{y0, z0}, h1, n1];
euler2 = eulerMethod[{y0, z0}, h2, n2];

ListLinePlot[
линейный график данных
 {
 euler1[[All, {1, 2}]], euler1[[All, {1, 3}]],
        всё                    всё
 euler2[[All, {1, 2}]], euler2[[All, {1, 3}]]
        всё                    всё
 },
 PlotStyle → {Red, Blue, Green, Purple},
 стиль графика  кра··· синий  зелёный фиолетовый
 PlotLegends → {"y(x) c h1=0.1", "z(x) c h1=0.1",
 легенды графика
  "y(x) c h2=0.05", "z(x) c h2=0.05"},

 PlotRange → All
 отображаемы··· всё
]
```

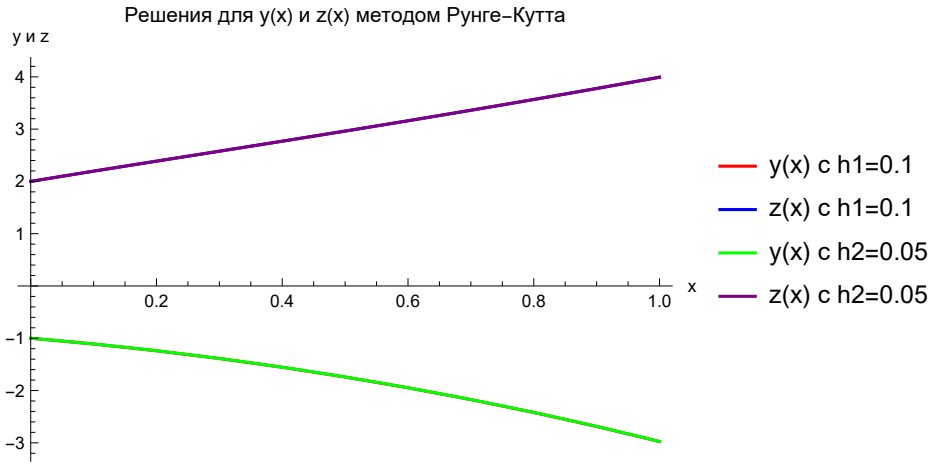
Out[•]=



б) Метод Рунге - Кутта

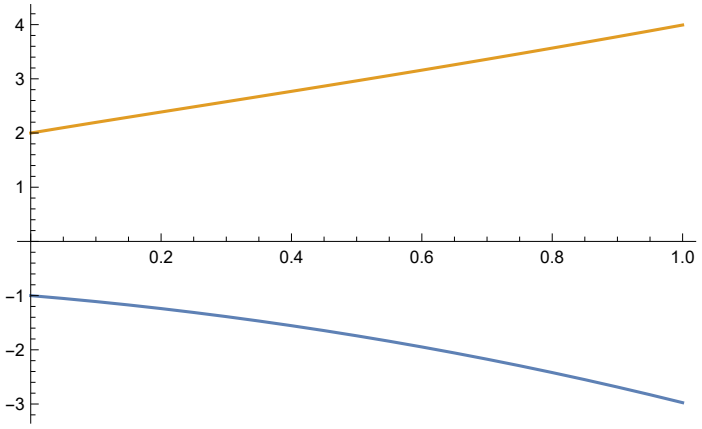
In[ ]:=
```
Clear[y, z]
очистить
a = 0;     b = 1;     y0 = -1;     z0 = 2;     h1 = 0.1;     h2 = 0.05;

f[y_, z_] := 1 - z;
g[y_, z_, x_] := Log[x + 1] - (2 * y) / ((x + 1) ^2);
                 натуральный логарифм

rungeKuttaMethod[{y0_, z0_}, h_, n_] :=
  Module[{x = a, y = y0, z = z0, result = {{x, y, z}}}, AppendTo[result, {x, y, z}];
  программный модуль                                    добавить в конец к
   For[i = 1, i ≤ n, i++,
   цикл ДЛЯ
    {k1y, k1z} = {h * f[y, z], h * g[y, z, x]};
    {k2y, k2z} =
     {h * f[y + 0.5 * k1y, z + 0.5 * k1z], h * g[y + 0.5 * k1y, z + 0.5 * k1z, x + 0.5 * h]};
    {k3y, k3z} =
     {h * f[y + 0.5 * k2y, z + 0.5 * k2z], h * g[y + 0.5 * k2y, z + 0.5 * k2z, x + 0.5 * h]};
    {k4y, k4z} = {h * f[y + k3y, z + k3z], h * g[y + k3y, z + k3z, x + h]};
    y = y + (1 / 6) (k1y + 2 * k2y + 2 * k3y + k4y);
    z = z + (1 / 6) (k1z + 2 * k2z + 2 * k3z + k4z);
    x = x + h;
    AppendTo[result, {x, y, z}]; ];
    добавить в конец к
   result];

n1 = Floor[(b - a) / h1];
     округление вниз
n2 = Floor[(b - a) / h2];
     округление вниз

rk1 = rungeKuttaMethod[{y0, z0}, h1, n1];
rk2 = rungeKuttaMethod[{y0, z0}, h2, n2];

ListLinePlot[{rk1[[All, {1, 2}]], rk1[[All, {1, 3}]], rk2[[All, {1, 2}]], rk2[[All, {1, 3}]]},
линейный график да···  всё                  всё                  всё                  всё
 PlotStyle → {Red, Blue, Green, Purple},
 стиль графика  кр···  синий  зелё···  фиолетовый
 PlotLegends → {"y(x) c h1=0.1", "z(x) c h1=0.1", "y(x) c h2=0.05", "z(x) c h2=0.05"},
 легенды графика
 AxesLabel → {"x", "y и z"},
 обозначения на осях
 PlotLabel → "Решения для y(x) и z(x) методом Рунге-Кутта", PlotRange → All]
 пометка графика                                            отображаем···  всё
```

Out[•]=



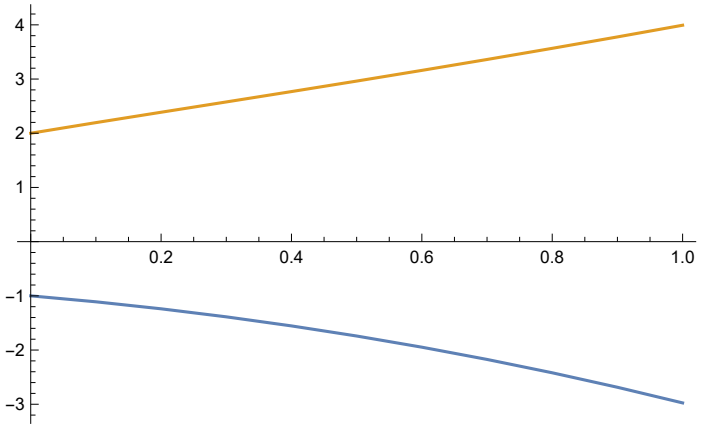
In[•]:=
```
ListLinePlot[{rk2[[All, {1, 2}]], rk2[[All, {1, 3}]]}, PlotRange → All]
```
линейный график да··· всё всё отображаем··· всё

Out[•]=



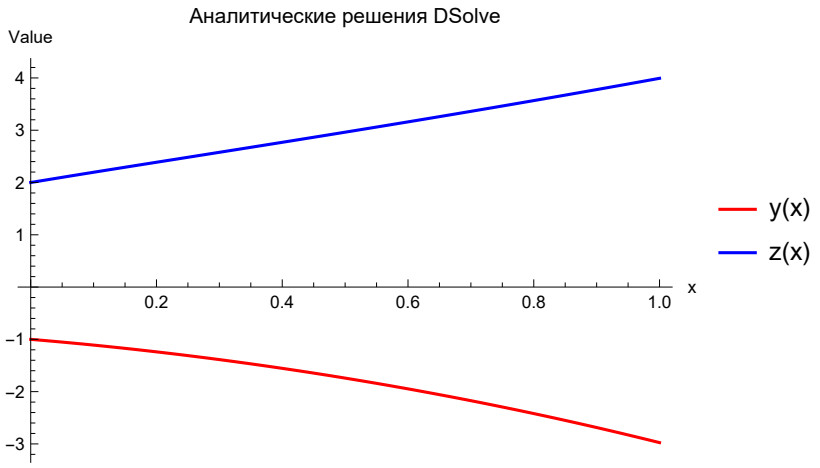
In[•]:=
```
ListLinePlot[{rk1[[All, {1, 2}]], rk1[[All, {1, 3}]]}, PlotRange → All]
```
линейный график да··· всё всё отображаем··· всё

Out[•]=

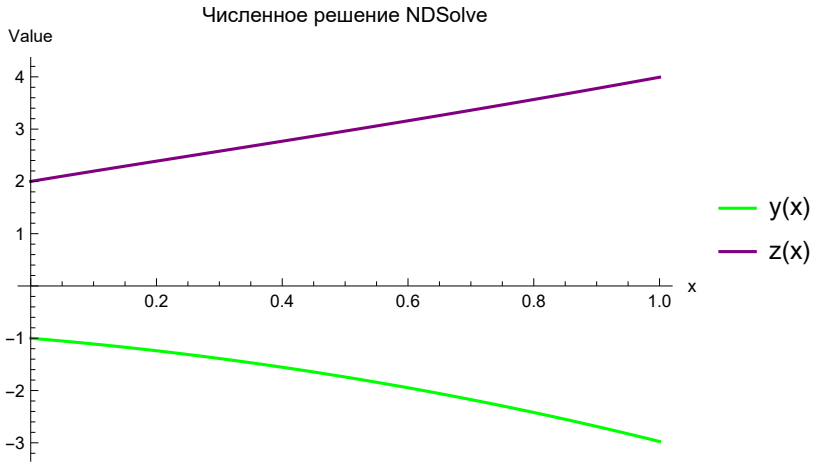


In[ ]:=
```
Clear[y, z]
очистить
eq1 = y'[x] + z[x] == 1;
eq2 = z'[x] + (2 y[x]) / ((x + 1)^2) == Log[x + 1];
                                        натуральный логарифм
initialConditions = {y[0] == -1, z[0] == 2};

solutionDSolve = DSolve[{eq1, eq2, initialConditions}, {y, z}, x];
                 решить дифференциальные уравнения
ySolution = y[x] /. solutionDSolve[[1]];
zSolution = z[x] /. solutionDSolve[[1]];

Plot[{ySolution, zSolution}, {x, 0, 1},
график функции
 PlotStyle → {Red, Blue}, PlotLegends → {"y(x)", "z(x)"},
 стиль графика  кр···  синий  легенды графика
 AxesLabel → {"x", "Value"}, PlotLabel → "Аналитические решения DSolve"]
 обозначения на осях          пометка графика                    решить дифферен

numericalSolution = NDSolve[{eq1, eq2, initialConditions}, {y, z}, {x, 0, 1}];
                    численно решить ДУ

Plot[{y[x] /. numericalSolution, z[x] /. numericalSolution},
график функции
 {x, 0, 1}, PlotStyle → {Green, Purple}, PlotLegends → {"y(x)", "z(x)"},
            стиль графика  зелё···  фиолет···  легенды графика
 AxesLabel → {"x", "Value"}, PlotLabel → "Численное решение NDSolve"]
 обозначения на осях          пометка графика                    численно решить ДУ
```

Out[ ]=



Out[•]=


Численное решение NDSolve
Value
4
3
2
1
−1
−2
−3
0.2
0.4
0.6
0.8
1.0
x
y(x)
z(x)